



Lab 05 – Worksheet

Name: Abdullah Rasool Arsalan Rashid	ID:ar12008 ar11014	Section: T2
---	-----------------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

**Task 1**

FSM:

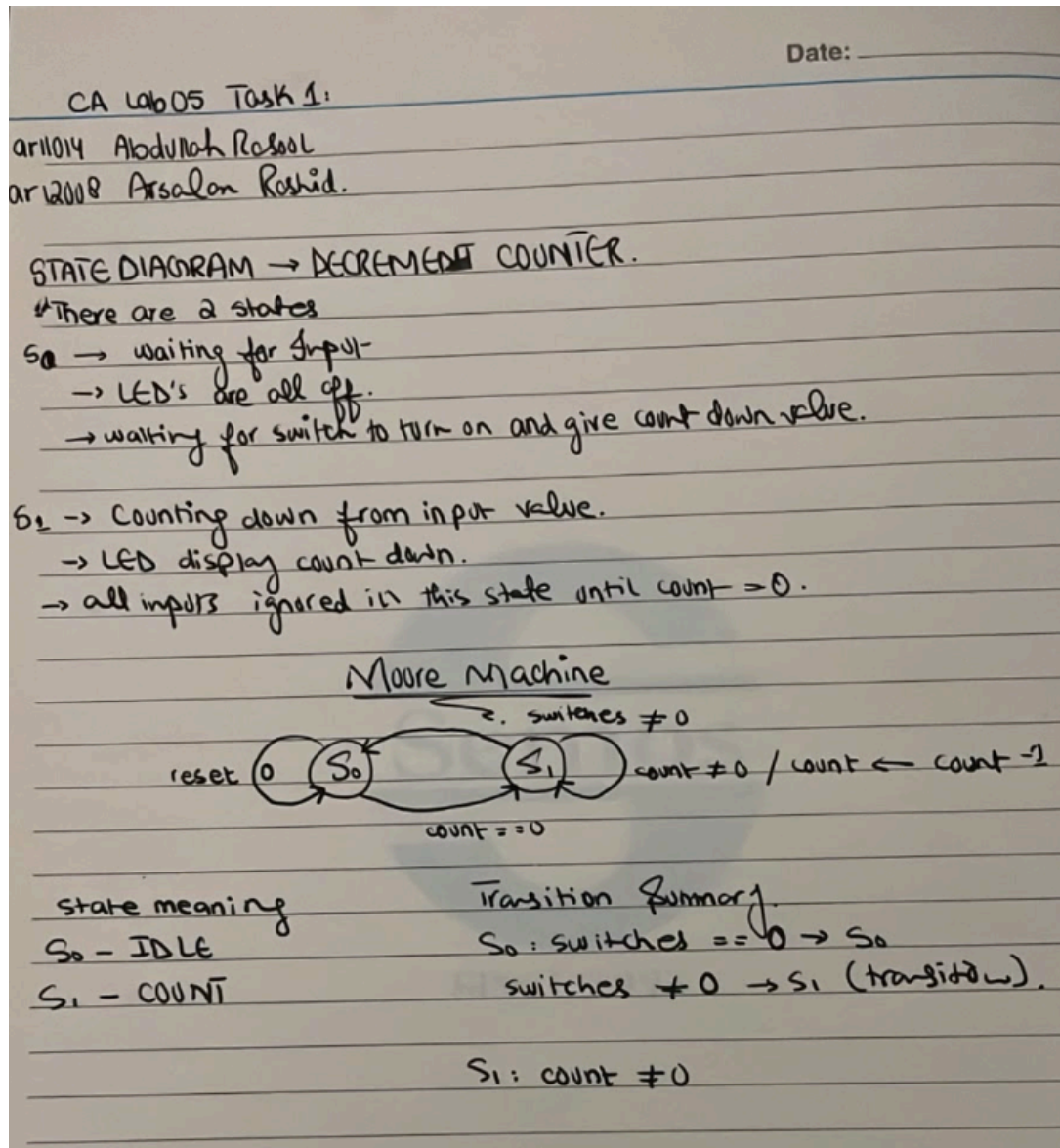


Figure 1: FSM state diagram (Moore machine) for the countdown controller



Comments

The implemented controller uses a two-state Moore Finite State Machine. In this design, the output signals are controlled only by the current state of the FSM rather than directly depending on the inputs. The two states separate the waiting operation and the countdown operation, making the behavior of the system easier to control.

State	Description	Counter Operation	LED Output
S0 – IDLE	The system waits until a non-zero switch value is provided	Counter remains disabled and stays at zero	LEDs remain OFF
S1 – COUNTDOWN	The stored switch value is displayed and reduced gradually	Counter decreases by one after every slow clock cycle	LEDs display current counter value

State Transitions

- **S0 → S0:** The FSM remains in the idle state when all switches are zero.
- **S0 → S1:** When a non-zero switch input is detected, the value of the switches is stored inside the counter and the FSM starts the countdown process.
- **S1 → S1:** During the countdown state, the counter continues decreasing on every slow clock pulse until it reaches zero. Any switch changes during this stage are ignored.
- **S1 → S0:** Once the counter reaches zero, the FSM returns back to the idle state.
- **Reset Operation:** The reset button works asynchronously. Pressing reset immediately clears the counter, turns off the LEDs, and returns the FSM to the idle state from any current condition.

Task 2

Modules + Testbench:

Written for this lab: **top_fsm_system.v** (FSM and counter), **clock_divider.v** (1 Hz slow clock; MAX_COUNT made a parameter so simulation can shorten it) and the testbench **top_tb.v**. The provided switches.v, leds.v and debouncer.v modules are used without modification.

top_fsm_system.v

```
`timescale 1ns / 1ps
```



```
module top_fsm_system (
    input wire clk,
    input wire pbin,
    input wire [15:0] physical_sw,
    output wire [15:0] physical_leds
);

    localparam IDLE = 1'b0;
    localparam COUNTDOWN = 1'b1;

    wire rst_clean;
    wire [31:0] switch_data;
    wire [31:0] led_write_data;
    wire slow_clk;

    reg state;
    reg [15:0] counter;

    assign led_write_data = {16'd0, counter};

    debouncer rst_db (
        .clk(clk),
        .pbin(pbin),
        .pbout(rst_clean)
    );

    leds switch_reader (
        .clk(clk),
        .rst(rst_clean),
        .btns(16'd0),
        .writeData(32'd0),
        .writeEnable(1'b0),
        .readEnable(1'b1),
        .memAddress(30'd0),
        .switches(physical_sw),
        .readData(switch_data)
    );

    switches led_writer (
        .clk(clk),
        .rst(rst_clean),
        .writeData(led_write_data),
        .writeEnable(1'b1),
        .readEnable(1'b0),
        .memAddress(30'd0),
        .readData(),
        .leds(physical_leds)
    );

    clock_divider ticker (
        .Clk_in(clk),
        .rst(rst_clean),
        .clk_out(slow_clk)
    );

    always @(posedge slow_clk or posedge rst_clean) begin
        if (rst_clean) begin
            state <= IDLE;
            counter <= 16'd0;
        end else begin
            case (state)
                IDLE: begin
                    if (switch_data[15:0] != 16'd0) begin
                        state <= COUNTDOWN;
                        counter <= switch_data[15:0];
                    end
                end
                COUNTDOWN: begin
                    if (counter != 16'd0)
                        counter <= counter - 16'd1;
                    else
                        state <= IDLE;
                end
            endcase
        end
    end

endmodule
```

**clock_divider.v**

```
`timescale 1ns / 1ps

module clock_divider (
    input wire clk_in,
    input wire rst,
    output reg clk_out
);

    parameter MAX_COUNT = 50_000_000 - 1;

    reg [25:0] count;

    always @(posedge clk_in) begin
        if (rst) begin
            count <= 26'd0;
            clk_out <= 1'b0;
        end else if (count == MAX_COUNT) begin
            count <= 26'd0;
            clk_out <= ~clk_out;
        end else begin
            count <= count + 26'd1;
        end
    end

endmodule
```

toptb.v

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/24/2026 09:14:11 AM
// Design Name:
// Module Name: toptb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:

module toptb ();

    reg clk;
    reg pbin;
    reg [15:0] physical_sw;

    wire [15:0] physical_leds;

    top_fsm_system uut (
        .clk(clk),
        .pbin(pbin),
        .physical_sw(physical_sw),
        .physical_leds(physical_leds)
    );

    defparam uut.ticker.MAX_COUNT = 2;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin

        pbin = 0;
        physical_sw = 16'd0;

        #10;
        pbin = 1;
        #20;
```



```
pbin = 0;
#20;
physical_sw = 16'd10;

#180;
physical_sw = 16'd0;
#40;
physical_sw = 16'd5;
#30;

pbin = 1;
#20;
pbin = 0;
physical_sw = 16'd0;

#50;
$finish;
end

endmodule
```

Results:

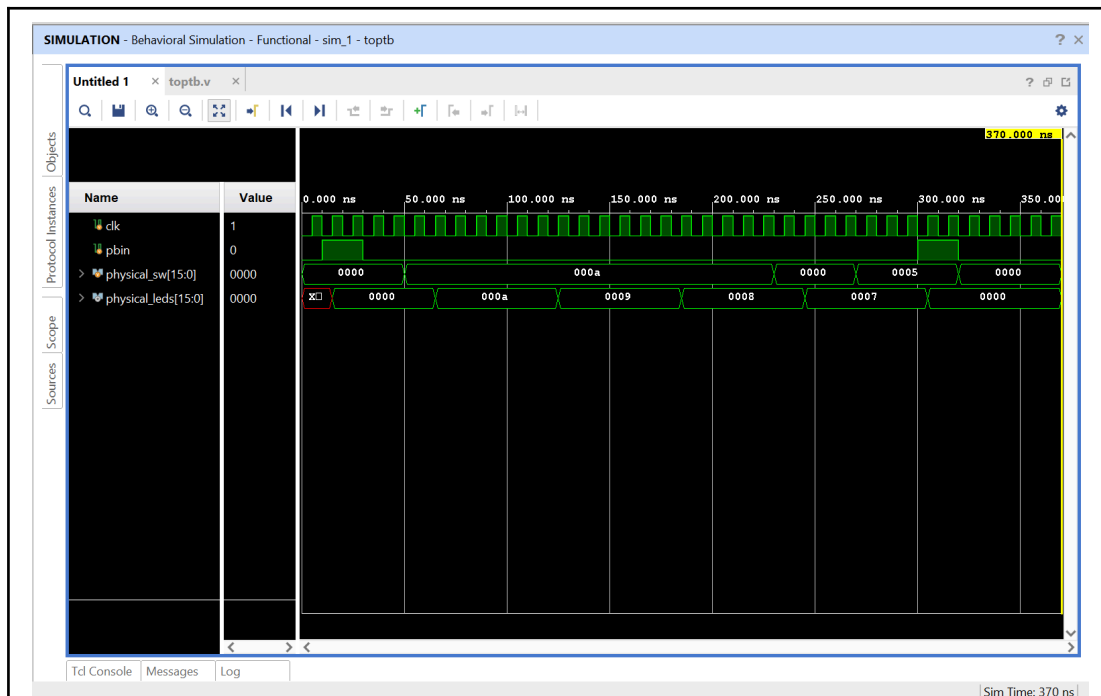


Figure 2: Behavioural simulation of toptb in Vivado XSim, 0–370 ns (values in hex)

Table 2: Simulation timeline

The design is divided into multiple modules to simplify the overall implementation. The main module, **top_fsm_system.v**, combines the FSM controller and countdown counter logic. The **clock_divider.v** module generates a slower clock signal required for the countdown operation, while the testbench (**toptb.v**) is used to verify the functionality through simulation.

The given supporting modules such as switch reading, LED writing, and debouncing were reused without modification. The clock divider parameter was



adjusted in simulation to reduce the waiting time and make the countdown behavior easier to observe.

The testbench applies different inputs including reset signals, switch values, and input changes during countdown to verify that:

- The initial reset condition works correctly.
- A non-zero switch value starts the countdown.
- Switch changes during counting do not affect the current countdown.
- Reset can interrupt the countdown and return the system to its initial state.

Task 3

XDC file

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk [get_ports clk]

set_property PACKAGE_PIN U18 [get_ports pbin]
set_property IOSTANDARD LVCMOS33 [get_ports pbin]

set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[15]}]

set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[7]}]
```



```
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[9]}]
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[15]}]
```

Timing Summary

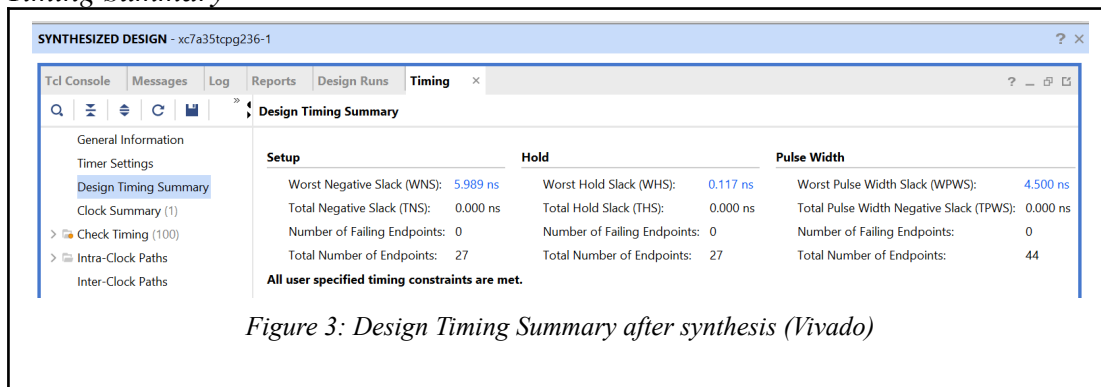


Figure 3: Design Timing Summary after synthesis (Vivado)

*Interpretation:*

Metric	After synthesis	After implementation
WNS (setup)	+5.989 ns	+5.908 ns
WHS (hold)	+0.117 ns	+0.289 ns
WPWS (pulse width)	+4.500 ns	+4.500 ns
Failing endpoints	0 of 27	0 of 27

Table 3: Timing results for the 100 MHz system clock (10 ns period)

Result: PASS. All slacks are positive and Vivado reports “All user specified timing constraints are met.” The slowest path takes about $10 - 5.989 \approx 4.0$ ns of the 10 ns period, a margin of about 60 %, so the design would still meet timing up to roughly 249 MHz ($1 / 4.011$ ns).

Limitation: the 17 FSM registers (state and the 16-bit counter) are clocked by `slow_clk`, which is generated by a flip-flop in the clock divider and has no clock constraint. Vivado therefore excludes them from the WNS (check_timing: no_clock = 17; methodology critical warning TIMING-17). At 1 Hz they have about 1 s to settle, so operation is not affected; a cleaner design would keep all logic on the 100 MHz clock and use a 1 Hz enable pulse.

Task 4

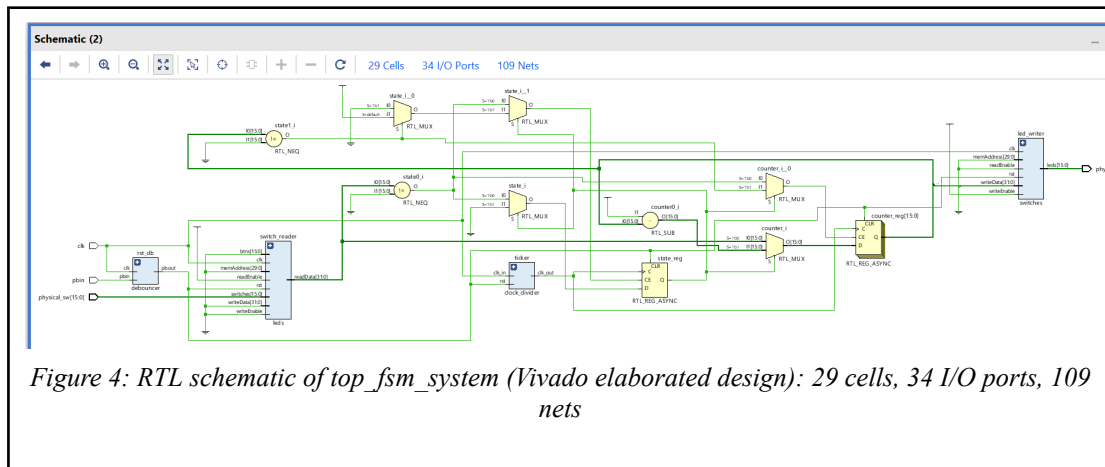
RTL Schematic:

Figure 4: RTL schematic of `top_fsm_system` (Vivado elaborated design): 29 cells, 34 I/O ports, 109 nets





Results

RTL analysis

- **Structure:** inputs clk, pbin and physical_sw[15:0] enter on the left and physical_leds[15:0] leaves on the right. The helper modules appear as blocks: rst_db (debouncer), switch_reader (leds), ticker (clock_divider) and led_writer (switches).
- **State transitions:** state_reg is an RTL_REG_ASYNC (flip-flop with asynchronous CLR from the reset) clocked by ticker's clk_out, the slow clock. Its next value comes from the RTL_MUX chain state_i, state_i_0 and state_i_1, driven by two RTL_NEQ comparators: state0_i checks switches $\neq 0$ (IDLE $\rightarrow$ COUNTDOWN) and state1_i checks counter $\neq 0$ (stay in COUNTDOWN or return to IDLE).
- **Counter:** counter_reg[15:0] is a 16-bit RTL_REG_ASYNC with clock enable (CE) and asynchronous clear. The RTL_MUX counter_i feeds its D input with either the switch value (load in IDLE) or the output of the RTL_SUB counter0_i (counter $- 1$). CE is active only when loading or decrementing, so the register holds its value otherwise.
- **LED path:** counter_reg drives writeData of led_writer, which registers it on the 100 MHz clock and drives physical_leds[15:0].

Resource (post-implementation)	Used	Available	Utilisation
Slice LUTs	47	20,800	0.23 %
Flip-flops	60	41,600	0.14 %
I/O pins	34	106	32.08 %
Global clock buffers (BUFG)	1	32	3.13 %

Table 4: Resource usage on the xc7a35tcbg236-1 (Basys 3)

The 60 flip-flops match the design: 26 (divider count) + 1 (slow_clk) + 1 (state) + 16 (counter) + 16 (LED register).

On-board test

The bitstream (top_fsm_system.bit) was generated and programmed onto the Basys 3 through the Hardware Manager. All tests passed:

Test	Action	Observed behaviour	Result
Zero input	All switches down	LEDs stay off; FSM waits in IDLE	Pass
Countdown	Set a non-zero value on the switches	LEDs show the value, then count down by one each second to 0; FSM returns to IDLE	Pass
Inputs ignored	Change switches during the countdown	Countdown continues unaffected	Pass
Reset	Press the centre button (U18) mid-count	LEDs clear immediately; FSM returns to IDLE	Pass

Table 5: On-board verification

Note: by design, if the switches are still non-zero when the count reaches 0, IDLE captures the value again and a new countdown starts.



Lab 05 – Designing FSM Using FPGA Switches and LEDs

Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Prelab		/15		/10 & /10
Task 1	-	-	/10	
Task 2	/15	/10		
Task 3		/15		
Task 4		/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.



AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.
-----	--------	--	--	--	---